

B.Sc. (Hons) in Software Development



Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

Stock Wise

By
Jennifer Oji

April 26, 2026

Minor Dissertation

**Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.**

Contents

1	Introduction	2
1.1	Financial Markets and Stock Portfolios	2
1.2	Machine Learning as a solution	3
1.3	Problem Area of project	3
1.4	Project Overview: StockWise	4
1.5	Objectives	5
1.6	Project Structure	6
2	Methodology	7
2.1	Project Approach and Rationale	7
2.2	Planning and Requirements	7
2.3	Project Management	8
2.4	Development Process	9
2.5	Validation and Testing	9
2.6	Technology Selection	9
2.7	Financial Analytics Methods	10
2.8	Data Sources and Preprocessing	11
2.8.1	News Sentiment	11
2.8.2	Stock Risk Clustering	11
3	Technology Review	13
3.1	React	13
3.2	Go	14
3.3	PostgreSQL	14
3.4	Python	15
3.5	Gaussian Mixture Model (GMM)	16
3.6	Linear Support Vector Classification (LinearSVC)	16
3.7	Scikit-learn	17
3.8	Pandas and NumPy	17
4	System Design	18
4.0.1	Overview of Project Architecture	18
4.0.2	Component interaction and coupling	19
4.0.3	Deployment and Cloud Architecture	19
4.1	Features and User Experience	20
4.1.1	User sign up	20
4.1.2	Stock holdings	22
4.1.3	News Sentiment	23
4.1.4	Analytics Script for Stock Features	27
4.1.5	Risk Metrics	28
4.1.6	Performance Metrics	29
4.1.7	Sector Allocation	31

4.1.9	Diversification Suggestions	34
4.1.10	Stock Risk	35
5	System Evaluation	38
5.1	Evaluation Against Objectives	38
5.1.1	Objective 1: Secure User Account Creation	38
5.1.2	Objective 2: Portfolio Upload	39
5.1.3	Objective 3: ML Diversification Suggestions	40
5.1.4	Objective 4: Stock News Integration	42
5.1.5	Objective 5: Risk Analytics and Feedback	43
5.1.6	Objective 6: Portfolio Performance Analysis	45
5.2	Strengths and Limitations of the Overall System	46
6	Conclusion	48
A	Appendix	51
A.1	Source Code Repository	51

List of Figures

1.1	Number of people moving money to investment accounts in 2025, sourced from [1]	4
2.1	Kanban board used for managing StockWise in Jira	8
3.1	Comparison of language response times in an environment with 100 concurrent users. Adapted from [2]	15
4.1	Technology Stack Used in StockWise	19
4.2	Sign up screen to enter email, password and risk tolerance	21
4.3	Diagram showing the user login process	21
4.4	Stock holdings page with example of a user's portfolio	22
4.5	Diagram of the portfolio retrieval flow	23
4.6	News page displaying financial articles with ML-classified sentiment label attached	24
4.7	Diagram of news retrieval from Finnhub API and sentiment classification	25
4.8	Stock Sentiment page showing the average sentiment score for each stock	27
4.9	Portfolio level risk metrics showing Volatility, Max Draw Down, Sharpe Ratio, and VaR calculations	28
4.10	Stock risk clustering categories and stock level risk metrics	29
4.11	Risk metrics flow	30
4.12	Portfolio Analysis page showing portfolio returns, best and worst performing stock, and holdings breakdown	30
4.13	Sector allocation page displaying the interactive sector pie and doughnut chart	31
4.14	Diversification suggestions flow	33
4.15	Example of the generated diversification suggestions in line with the user's selected risk preference	34
4.16	Diversification suggestions flow	35

4.17	Enter Caption	36
4.18	Simulating the impact of 22 AAPL shares on this specific portfolio .	37
5.1	Stock risk clusters from the GMM model output	41
5.2	Confusion Matrix of sentiment classification model	44

List of Tables

5.1	Secure Account Creation Acceptance Tests	39
5.2	Portfolio Upload Acceptance Tests	40
5.3	Best Model per Covariance Type	41
5.4	Stock Risk Cluster Interpretation	42
5.5	Classification Report for LinearSVC	43
5.6	Test portfolio composition	44
5.7	Expected vs. actual portfolio values	45
5.8	Test portfolio composition	46
5.9	Manual vs system portfolio performance results	46

Chapter 1

Introduction

1.1 Financial Markets and Stock Portfolios

The stock market is an online marketplace where investors can buy and sell shares of publicly listed companies. Owning shares of a company means owning a small portion of it. When the company's value grows or declines, so do those shares. A stock portfolio is the set of stock holdings an investor owns. Creating and managing a stock portfolio helps investors achieve their long-term financial goals.

There are many investment strategies used to optimise a stock portfolio, the choice of which depends on the goals the investor hopes to achieve from it. Diversification is a commonly used approach to portfolio optimisation. The purpose of diversification is to spread stock allocations across different sectors - such as technology, consumer goods and healthcare, in order to reduce exposure to any single area. For example, if an investor's portfolio only contained technology stocks like Apple or Microsoft, any event that would negatively affect the technology sector would cause losses across the portfolio, as it was only composed of technology assets. Diversification prevents against this risk by spreading holdings across many sectors.

The need for portfolio optimisation stems from the fact that stock markets are incredibly volatile as they are influenced by numerous, unpredictable factors such as investor sentiment, political events and global happenings [3]. However, investors can protect themselves from the fluctuations in market conditions by managing risk. Portfolio optimisation enables an investor to be profitable while keeping risk at a level they are comfortable with.

1.2 Machine Learning as a solution

Portfolio optimisation was first introduced by [4] with Mean Variance Optimisation (MVO), which is an approach that says investors should create portfolios that generate the maximum returns for a certain level of risk or minimise risk for a given level of return. However, MVO often struggles to perform well in complex markets due to its reliance on assumptions. This leads to poor performance on data it was not trained on [5], so it is a relatively unadaptive approach. Machine Learning is another way to optimise portfolios that is better aligned with the versatility of stock markets.

Machine learning (ML) is a branch of Artificial Intelligence (AI) that allows computers to take in datasets as input and learn from them in such a way that allows them to generate outputs without the rules needed to produce them. There are many different types of ML algorithms, each with its own set of strengths for different sets of tasks. The main types of ML algorithms are supervised learning, unsupervised learning and reinforcement learning.

ML models can be used to analyse historical stock data and trends to help decision making in portfolio construction. A commonly used ML approach is clustering, which is a form of unsupervised learning. Clustering is used to group similar data points into groups based on their characteristics and not their predefined labels. Clustering can analyse patterns in datasets to group stocks that have similar risk and return together, essentially grouping them based on their performance in the market and not their sectors. This promotes diversification for portfolio optimisation [6].

1.3 Problem Area of project

The Federal Reserve collects data on United States household finances every three years to produce the survey of consumer finances. In the year 2022, this survey revealed that stock ownership reached an all time high, with about 58% of U.S families holding some form of equity[7]. This reflects the growth in retail investing over the decades.

This growth is also reflected in [1], which shows retail investing activity in 2025 is much higher in terms of both participation and amounts invested since 2015, as shown in Fig 1.1.

However, many of these retail investors will be relatively inexperienced, without a strong background in finance. This may lead to them making investment decisions based on gut feeling, social media trends, or even hype rather than data backed decisions, which exposes them to unnecessary risk. This motivates the need for an accessible and beginner-friendly portfolio insight tool to aid inexperienced

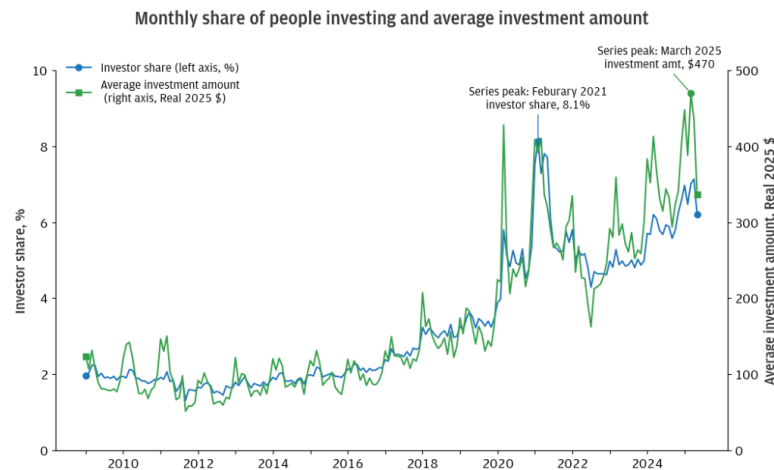


Figure 1.1: Number of people moving money to investment accounts in 2025, sourced from [1]

retail investors in the stock market.

1.4 Project Overview: StockWise

StockWise is a full-stack Web Application that is designed to help beginner retail investors get a better understanding of their portfolio's performance, risk profile, diversification needs, and general market context. It uses financial analytics and machine learning to convert the user's portfolio data into actionable insights, tailored to their risk preference.

The application has three technology components: a React frontend deployed as a Progressive Web App (PWA), a Go backend API that acts as the application server, and a Python service layer which is responsible for all ML computation and portfolio analysis. The services are also containerised with Docker for deployment and Docker Compose for running locally.

StockWise uses two machine learning processes: a Gaussian Mixture Model (GMM) for stock risk clustering, producing six risk bands. And Linear Support Vector Machine (LinearSVM) for news sentiment classification.

The main features of the application are:

- Secure account creation with risk preference selection (Low, Moderate, or High Risk).
- Portfolio holdings management, for adding, editing, and deleting stocks.

- A performance analytics dashboard showing portfolio and individual stock returns.
- A risk metrics dashboard showing volatility, VaR, and maximum drawdown.
- Stock risk classification, for users to understand the risk category of their individual holdings.
- Sector allocation visualisation through interactive charts.
- ML driven diversification recommendations aligned to the user's selected risk preference.
- A stock risk checker enabling users to assess any stock tickers' risk.
- Simulation of adding a new stock to the portfolio to show its effects before purchasing.
- Stock news and sentiment to gauge the current market mood towards the user's current holdings.

1.5 Objectives

The goal of StockWise is to build a financial tool accessible across multiple devices for inexperienced retail investors to make smarter, data-driven decisions, instead of relying on emotional factors. The success of StockWise will be evaluated in Chapter 5 against the following objectives.

1. **Secure User Account Creation:** Allow the user to securely create an account using an email, password and a selected risk preference of Low, Moderate or High Risk. *Success metric: users can register, log in, and have their risk preference consistent in the application.*
2. **Portfolio Upload:** Enable users to upload and manage their current stock holdings, including adding, editing, and deleting entries. *Success metric: portfolio is accurately stored and is reflected in all features.*
3. **Machine Learning-Based Diversification Suggestions:** Use ML model to analyse the user's portfolio to generate diversification suggestions tailored to their specified risk preference. *Success metric: recommendations are only produced from the appropriate risk cluster and exclude stocks the user already holds.*

4. **Machine Learning News Sentiment Analysis:** Use ML to classify news articles with sentiment labels such as Positive, Negative or Neutral. *Success metric: sentiment labels are generated for each news article and displayed alongside the relevant article.*
5. **Risk Analytics Dashboard:** Calculate and display the key risk metrics based on the user's stock holdings, including measures such as volatility, max drawdown and Value At Risk (VAR) *Success metric: metrics are computed correctly from the user's holdings and displayed.*
6. **Portfolio Performance Analysis:** Analyse the user's current portfolio to compute performance metrics, such as returns. *Success metric: performance metrics are accurately computed and displayed*

1.6 Project Structure

This dissertation is laid out in the following way. Chapter 1 discusses the background, motivation and objective of the project. Chapter 2 describes the methodology and the approach taken to develop the project. Chapter 3 describes each of the technologies used and why they were chosen. Chapter 4 covers the system design and project architecture. Chapter 5 evaluates the final application against each of the objectives set out above. The source code for this project is available on **GitHub** (see Appendix A.1).

The repository is organised into three main directories. The **backend** folder contains the Go application used for orchestration, connecting all of the components of the system. The **frontend** folder contains the React application that provides the user interface of StockWise. The **services** folder contains the Python FastAPI service used to provide the data-driven insights, along with the scripts used to build and train the GMM and LinearSVM model. The README provides an overview of the project and all setup instructions to run the full application locally using Docker Compose.

Chapter 2

Methodology

2.1 Project Approach and Rationale

To develop StockWise, I used an incremental agile approach. I chose this for my project as it offers flexibility in the development process, instead of following a rigid timeline with little room to adapt. Iterative development allowed me to implement and test individual components and there was always a working prototype. Once I completed a component, I would then further refine it once feedback was received.

My research for this project was very user-focused and ensured that each feature, such as the risk metrics, diversification suggestions, and actionable insights in StockWise, were valid and supported the goal of helping inexperienced retail investors make smarter investment decisions.

2.2 Planning and Requirements

Once I solidified the project idea, I identified the necessary requirements for the application. I also examined existing platforms for investors and identified gaps I could build upon in StockWise to set it apart from current products in the market. To gather further requirements, I also consulted a stock trader on additional features they would find beneficial, which helped me design the app through the lens of a trader. The feedback given by the investor led me to create the feature of a stock-specific risk checker and a portfolio simulation.

I created user stories for the main application screens, based on the prototype, which helped me understand the behaviour of components and prioritise essential features. To do this, I split the project requirements into three categories: must have, should have, and nice to have, which allowed me to figure out what to focus on and what could wait until later. This method allowed me to concentrate on the important features first.

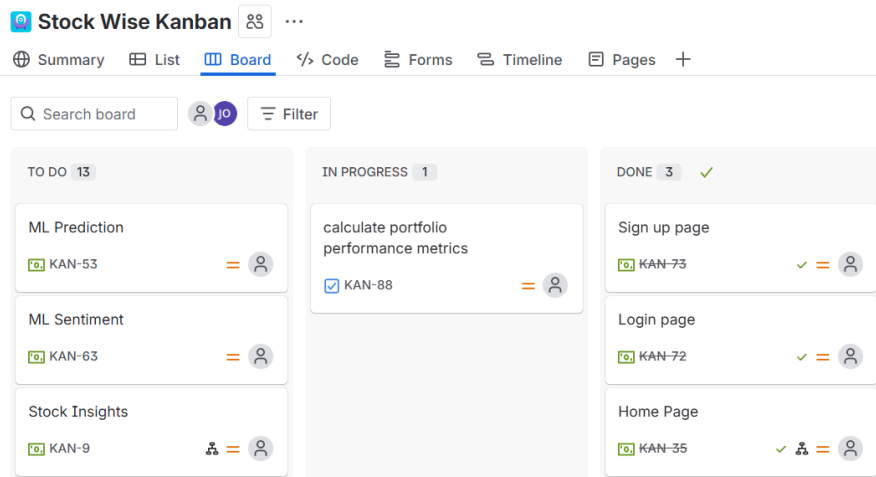


Figure 2.1: Kanban board used for managing StockWise in Jira

When the project requirements were fully defined, I created a low-fidelity prototype to help visualise the overall flow and the main features of the application. By designing a low-fidelity prototype, I was able to get a better idea of the overall flow from a user's point of view and visualise how each component would interact with each other. It also gave me clarity in the structure and layout of the app before any code was even written. Creating a prototype also reduced the risk of having to make any major design changes later on in the development process.

2.3 Project Management

For feature and task management, I used kanban boards on the platform Jira to organise and plan work throughout the project. Each feature, such as the stock insights screen, was broken down into smaller and more manageable tasks to create a smooth development process. For example, the news sentiment feature was split into tasks like - extract relevant news from Finnhub API, apply ML inference for sentiment analysis, integrate the API endpoint, and create the UI component. Splitting each feature into smaller tasks sped up the development process, as I always knew what needed to be implemented next. The kanban board used in StockWise is shown in Fig 2.1.

Throughout the project, I had weekly reviews to keep on track with the timeline I set out. In these reviews, I would assess my current progress, update the Kanban board, and set new goals for the week on what needed to be completed. Along with these weekly reviews, I would also periodically meet with the stock trader. The key meetings were after the initial prototyping and towards the end of the project

development for final validation. These meetings would consist of showing the current state of the application and discussing whether or not the implementation met traders' needs, as well as the quality of the user experience. The Feedback was then used to further refine the implementation. For example, I received feedback to add metric definitions to the risk metrics dashboard after the trader pointed out that terms like Value at Risk or Max Drawdown might not be familiar to a new investor.

I used GitHub for version control throughout the development process to frequently commit changes to the application. The GitHub repository was also linked to Jira so that closing a task on GitHub automatically marked it as complete in the kanban board.

2.4 Development Process

I first began the development process by creating the main architecture of the application. This involved setting up the React frontend, Go backend API, PostgreSQL database and the Python service to ensure they were communicating correctly, as all main features depend on these components working together.

When this was implemented, I followed the features and tasks on Jira, starting from the must-have features I defined. Each feature was built step by step, following the same structure: I would implement the feature in the FastAPI service, create an endpoint for it in the backend, create its UI component and then manually test it to ensure it worked.

2.5 Validation and Testing

Each piece of functionality was tested after it was completed, using acceptance testing to ensure it worked correctly in all scenarios. For example, the portfolio management feature was tested with inputs, such as null values and empty ticker symbols. This revealed errors early in development and allowed me to fix them as they arose instead of testing the application at the end of the development lifecycle.

Validation testing was also carried out during the review meetings with the stock trader to ensure the application met the end users needs and requirements.

2.6 Technology Selection

The technologies that were chosen were due to their suitability to complete a task. React and Expo were chosen as the frontend as it supports both mobile and web

development from a single codebase, fulfilling the requirement to be accessible on all device types. Go was chosen for the backend due to its efficiency and Speed. Python was chosen as the service layer for analytics as it is the language most commonly used for data processing and machine learning.

2.7 Financial Analytics Methods

Various metrics are used in finance to analyse a portfolio's risk.

Portfolio Volatility:

Portfolio volatility measures how much an asset's price swings over a given period of time. It's computed using the standard deviation of the portfolio returns and the square root of the portfolio variance [8]:

$$\sigma_p = \sqrt{\text{Var}(R_p)} \quad (2.1)$$

where $\text{Var}(R_p)$ is the variance of the portfolio returns.

Value at Risk (VaR):

Value at Risk is the maximum loss a portfolio is expected to take over a certain period of time, with a confidence level [9]:

$$\text{VaR}_\alpha = z_\alpha \sigma_p - \bar{R}_p \quad (2.2)$$

where z_α is the confidence level α , σ_p is the portfolio volatility, and $\bar{R}_p$ is the expected return of the portfolio.

Sharpe Ratio:

Sharpe ratio calculates the portfolio returns for the level of risk taken on [10]:

$$\text{Sharpe Ratio} = \frac{R_p - R_f}{\sigma_p} \quad (2.3)$$

where R_p is the portfolio return, R_f is the risk-free rate, and σ_p is the portfolio volatility.

Maximum Drawdown (MDD):

Maximum Drawdown is the distance between the peak value of a portfolio or stock and its lowest point over a certain period of time [11] .

$$\text{MDD} = \frac{T - P}{P} \quad (2.4)$$

where P is the peak portfolio value and T is the trough value after that peak. These risk calculations are used in StockWise to provide the performance and risk insights.

2.8 Data Sources and Preprocessing

2.8.1 News Sentiment

The data used to train the sentiment classification model was sourced from Kaggle from [12] and [13]. These datasets were combined and had a total of 10,159 news headlines and sentiment labels, however the dataset was imbalanced and contained more neutral classes than positive and negative. To combat this, the dataset was engineered to have equal parts of each sentiment label so it would be balanced, resulting in 8,299 headlines. So the shape of the final dataset used for the news sentiment pipeline is (8299, 2).

Once the sentiment data was loaded, the following steps were taken to train the model:

1. The dataset was split into training and test sets with a 70/30 split.
2. The headlines were processed to remove notes that would not be useful for sentiment classification, such as tags and URLs.
3. The headlines were converted into numbers using CountVectorizer and TF-IDF.
4. The LinearSVC was trained on the headlines and its associated sentiment.

The trained LinearSVC was then exported in ONNX format, which allows efficient inference in the FastAPI service. The model was converted to ONNX format as it transforms the model into a portable format that can run anywhere and makes it cheaper to run than querying raw Python ML code. Using ONNX also improves response times, so the application can efficiently provide insights in real time.

2.8.2 Stock Risk Clustering

The Gaussian Mixture Model (GMM) was used for stock risk clustering. This involved creating a stock universe for model training comprised of stocks with various known risk levels, from stable stocks to speculative ones.

Once this universe was defined, the following steps were taken to train the model:

1. One year of adjusted close prices was downloaded for each stock from Yahoo Finance.
2. Its Daily returns, annualised mean returns and annualised variance were calculated.

3. Risk metrics such as volatility, variance and VaR were computed from this data and are used as input features.
4. Bayesian Information Criterion (BIC) was used to find the most suitable number of clusters for the data (6).
5. The model was trained on these input features and assigned a risk cluster for each stock from Very Low to Extreme Risk.

A separate stock universe was defined for use in the application, comprised of 503 stocks from the S&P500 index, sourced from [14].

Chapter 3

Technology Review

3.1 React

React is a library used to create frontend applications like websites. Javascript is a dynamically typed programming language that manipulates the Document Object Model (DOM). This allows web content to be dynamically updated [15],[16]. JavaScript essentially allows markup language to be transformed into its visual representation and React then renders the HTML to display it in the DOM.

React's main architecture is component-based, which promotes a maintainable design as each component can be in charge of managing its own state and logic [15].

React can communicate with backend technologies through Fetch API, which can be used to send and retrieve data across the network [17].

React's component based architecture works well with the architecture and design of StockWise.

React is a library that utilises JavaScript to build user interfaces. It renders the application using the virtual DOM. This is an in-memory version of the JavaScript representation of the main Document Object Model (DOM), which displays HTML onto a web page. When React notices a change in the state of the application, it updates the virtual DOM and then compares the updated virtual DOM to the previous virtual DOM using a process called diffing, which spots the differences. Once the change is found, it updates the real DOM to reflect these changes [15].

React's main architecture is component-based, making it easy to maintain and scale as each component can be in charge of managing its own state [15]. This modular design is suitable for StockWise as components can be reused across different parts of the application, and it works well with its architecture.

React can communicate with backend technologies through HTTP using HTTP clients such as axios, which can be used to send and retrieve requests and responses

over the network [18]. This is critical for the functionality of StockWise as back-end communication is required to enable data holding and receiving generated analytics.

Expo Go was also used throughout the development process, which allowed the application to be debugged and tested directly on a mobile device before it was deployed. This helped identify bugs that might have gone unnoticed when debugging on the web alone.

3.2 Go

Go is a statically typed and compiled programming language. It is compiled into machine code, which lets programs execute with low run times, leading to improved performance. Go is still a relatively easy language to work with because of its simple syntax, despite being compiled [19].

Go's performance speed is a huge factor in why it was used in StockWise. Its capabilities are discussed in research from [2], which compared Go, PHP, Python, and Node.js for building API services in the cloud. Go and Node.js both performed well under low user concurrent conditions, but Go performed best when user concurrency was increased. See Fig 3.1 for the outcome of the research carried out. This means that with both low and high platform engagement go still had a relatively fast execution speeds compared to the other languages.

Iris is a Go framework used for HTTP, routing and middleware. I chose this over other frameworks, such as Gin and Echo because of its vast feature set. For StockWise, iris was used to receive requests from the React frontend and route them to the appropriate endpoints, such as the FastAPI service. Iris's middleware was also used for handling the CORS configuration for all the endpoints.

3.3 PostgreSQL

PostgreSQL is an object-relational database that uses the SQL language, along with many other features to safely store complex data in structured tables [20]. The data captured in StockWise has a naturally relational structure since a user has a portfolio, a portfolio has holdings, and each holding has purchases.

PostgreSQL also follows the ACID principles. **Atomicity** means that if any part of the whole transaction fails the whole process completely rolled back. **Consistency** means that all transactions must leave the database in a clean state **Isolation** means that all transaction are isolated from each other, preventing dirty reads **Durability** means that once data is committed, it becomes permanent, even if the database were to crash, the data persists [?].

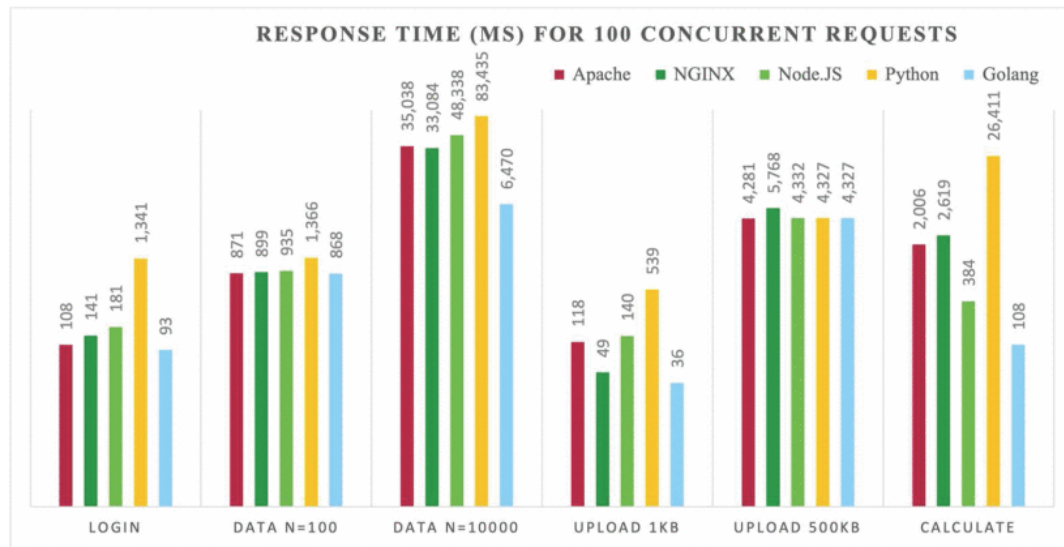


Figure 3.1: Comparison of language response times in an environment with 100 concurrent users. Adapted from [2]

These principles ensure that each operation is always processed safely. As StockWise is a financial application, data integrity is essential to the performance quality because incomplete writes to the database could lead to incorrect risk calculations being performed, which would lead to displaying the wrong information to the user [?].

3.4 Python

Python is an object-oriented and interpreted language used for application development. An interpreted language reads and executes code line by line. As seen in Fig 3.1, this makes Python have slower execution speed when compared to other languages [2].

Python has become the standard language for machine learning due to its extensive built in libraries for it. This includes Scikit-learn, pandas and NumPy which can be used to format data and train ML models on. This data handling convenience makes Python an optimal choice for ML projects.

Python was chosen for the microservice layer in StockWise because its native libraries are required to build and train the GMM clustering model and the LinearSVM sentiment model. Implementing ML in a different language would require

porting libraries, which would add unnecessary complexity. Its standard libraries like NumPy are used for the financial calculations in the application.

FastAPI is a framework that uses Python to create APIs for over-the-network communication. This was used in StockWise to send the computed financial analytics to the Go backend once requested.

3.5 Gaussian Mixture Model (GMM)

Clustering is an unsupervised machine learning approach used to group similar data points together, based on their characteristics and not their predefined labels. Unsupervised learning refers to models that are trained on data without labelled outcomes so the model needs to identify the patterns itself to group them. Using clustering was an appropriate approach for this project as there is no widely available dataset of stocks with labelled risk categories such as low, moderate or high risk.

The clustering algorithm used in StockWise to assign risk clusters to stocks is the Gaussian Mixture Model (GMM). GMM is a probabilistic clustering model that assumes data is not just coming from one group, but multiple overlapping ones. It performs soft clustering by assigning a probability of each data point belonging to multiple clusters instead of hard assignments. To do this, the GMM estimates three parameters: the mean, which represents where each gaussian should be centered; the covariance, which decides how spread out each cluster should be and the shape it should have; and the mixture of weights provided to each gaussian, which shows how much each gaussian contributes to the overall model [21]. This approach makes it suitable for financial data, as stock risk categories are not often cleanly separated [22].

3.6 Linear Support Vector Classification (LinearSVC)

Supervised learning is when a machine learning model is trained on labelled data. The ML algorithm essentially learns patterns from known pairs of data in training and uses the learned pattern recognition to predict values on unseen data.

LinearSVC is a supervised machine learning algorithm that uses support vectors for classification tasks. Even though it is typically used for binary classification, it supports multi-classification through the 'one-versus-rest' method, which essentially compares one class against all others [23]. For example, with sentiment classes with labels of positive, negative or neutral, the model creates three separate classifiers for each.

- negative vs (positive and neutral)

- neutral vs (negative and positive)
- positive vs (negative and neutral)

For each label, the model produces a classification score, and the label with the highest score is the one selected as the predicted class [23].

LinearSVC was used in StockWise to classify articles as positive, negative or neutral. It was chosen for this task as it works well on structured data and is compatible with ONNX for model inference [24]

3.7 Scikit-learn

Scikit-learn is an open-source Python library used for both supervised and unsupervised machine learning. It also provides a wide range of features for data processing, model fitting, and model evaluation [25],[26].

Scikit-learn is widely used in the industry as it offers a consistent way to build, train, and assess machine learning models.

Scikit-learn was used in StockWise as its vast feature set allowed the implementation of the GMM for stock risk clustering and the LinearSVC model for sentiment classification [27].

3.8 Pandas and NumPy

Pandas is an open-source library for working with datasets in Python. It can clean, analyse, and manipulate data, making it a useful tool for feature engineering. It is commonly used in machine learning during the preprocessing stages to prepare the data before the model is trained on it.

Pandas was used in StockWise for this reason, to read in and shape the stock data from the sourced dataset and to handle valid or missing values. Pandas was a suitable choice for this as it is a standard library in Python for handling data, and also works well with the rest of the machine learning pipeline.

NumPy is an open-source python library that provides numerical computing features. It is useful for working with high dimensional arrays and has features for machine learning applications.

It was used in StockWise to compute the mathematical calculations for stock risk and performance metrics features in the application.

Chapter 4

System Design

4.0.1 Overview of Project Architecture

StockWise is organised with a layered architecture that separates the React frontend, Go backend API, the Python FastAPI service, and the PostgreSQL database. This ensures scalability and maintainability through the separation of concerns.

Each part of the application communicates with each other through APIs and not dependencies, so the components do not directly affect each other. The function of the backend is to route and connect the relevant resources with each other, essentially acting as a bridge. The database stores users' data and stocks in PostgreSQL. And the FastAPI service is in charge of computing the portfolio analytics and connecting to external APIs. See Fig 4.1 for an overview of the architecture.

The **frontend** is built with React. It provides the user interface to interact with the application. It contains screens for creating an account, logging into an existing account, adding stock holdings, viewing relevant stock news, and portfolio insights. The contents of the frontend folder are also organised to follow React best practices and are structured to have a component-based design for reusability. This design decision makes the architecture easy to navigate.

The **backend** is built with Go as a RestAPI and its function is to communicate with all endpoints of the application. It is in charge of retrieving user data from the database, which enables a user to log in or save new stocks. It also sends the users' data to the FastAPI service to retrieve the generated insights based on their stock holdings.

The **service** uses Python FastAPI to communicate across the network. This component contains the ML models created to classify stock risk, produce sentiment labels from news headlines and generate portfolio insights.

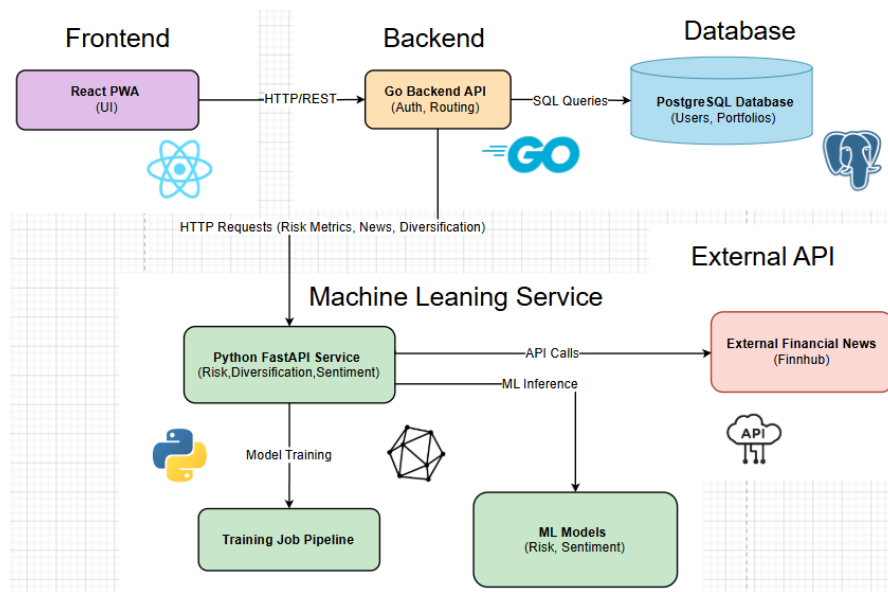


Figure 4.1: Technology Stack Used in StockWise

4.0.2 Component interaction and coupling

The architecture of this project follows the client-server architecture where the client is responsible for making requests and the backend is responsible for fulfilling them. This system design allows for a clear separation of concerns between the different components, as the frontend acts as the interface for users to interact with, while the backend is in charge of handling communication between other services, such as the FastAPI service and database. All components also work independently of one another, which promotes a scalable architecture, as updating one component does not directly affect the others in the application.

The components are also loosely coupled as communication is achieved through APIs and not direct dependencies. The frontend does not depend on the backend logic and only uses the responses returned from the API. The FastAPI services are also independent from the backend and each piece of functionality is separated into its own endpoint.

4.0.3 Deployment and Cloud Architecture

StockWise is deployed as a Progressive Web Application (PWA). This addresses the objective of creating an accessible platform, as users can use the application on Android, iOS, or a regular web browser.

The frontend of the application is deployed on Vercel, which is a cloud platform that is designed specifically for frontend development and hosting [28]. This makes

it a suitable platform for delivering the React user interface efficiently.

The Go RestAPI and the Python FastAPI are deployed on Render. Render is a unified cloud platform that supports full-stack application deployment, and APIs, background services, and databases [29]. The PostgreSQL database is also hosted on Render, along with a scheduled cron job which is responsible for computing and updating the daily stock features that are used by the machine learning components of the system.

4.1 Features and User Experience

4.1.1 User sign up

When a user first signs up, they are prompted to enter a username, email and password. Along with this, they must also select a risk tolerance. The risk tolerance is necessary for the risk-aware insights and portfolio optimisation suggestions. The user can select between the following risk preferences: Low risk, a stock portfolio that moves conservatively with both returns and risk. Moderate risk, a balanced portfolio with moderate risk and returns. High risk, a portfolio with high volatility and the possibility of higher returns. Fig 4.2 shows this feature.

The authentication process is responsible for verifying a user's details to give them access to the application. Their password is hashed and salted before it is stored in the database, and logging in verifies that the input hash matches the one in the database under the same email.

When a user tries to log in, the system follows this sequence of interactions, as shown in Fig 4.3:

1. User enters their credentials (email and password) along with a risk tolerance in the frontend interface.
2. Frontend sends a POST request with these credentials to the backend API.
3. Backend validates the request and checks the credentials match those in the PostgreSQL database.
4. If the credentials are valid, the backend responds with the user's information (ID, Username, Email, and Risk).
5. Frontend receives the user information and allows access to the application.



Create account

Set up your profile and choose your risk level.

Name

Email

Password



Risk tolerance

Select a risk level

Sign up

Back to Home

Figure 4.2: Sign up screen to enter email, password and risk tolerance

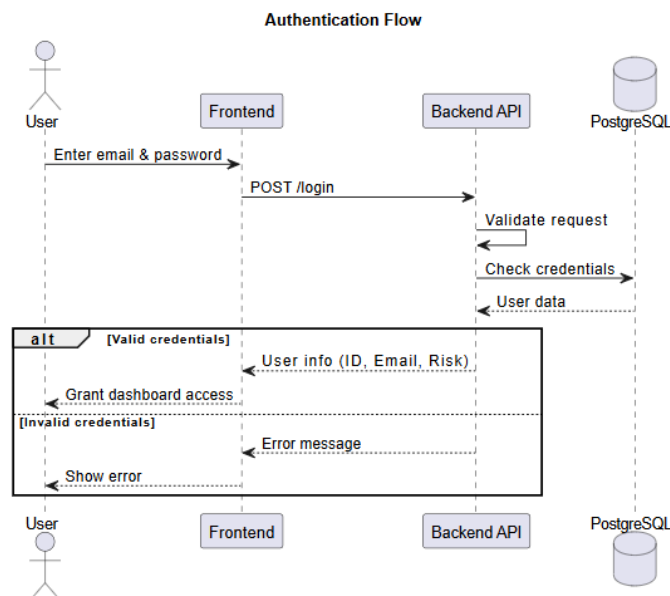


Figure 4.3: Diagram showing the user login process

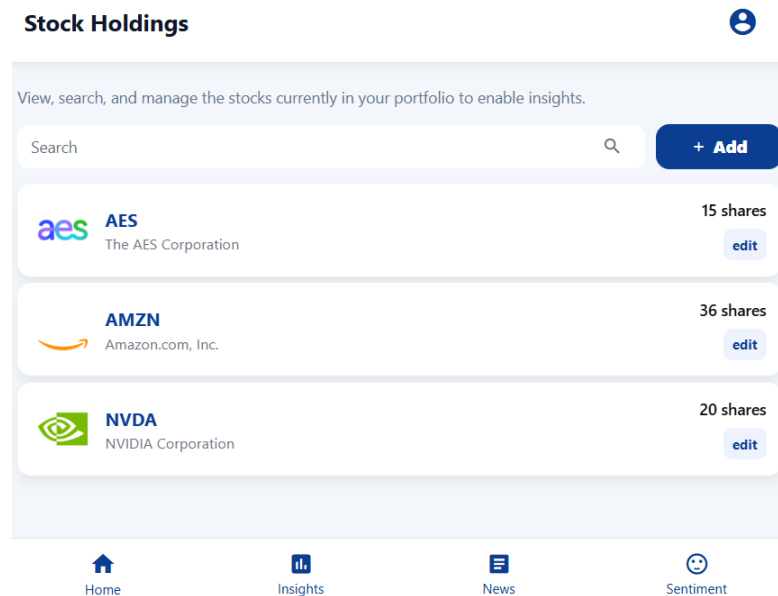


Figure 4.4: Stock holdings page with example of a user's portfolio

4.1.2 Stock holdings

Once the user is successfully authenticated, they are navigated to the stock holdings page, which allows the user to enter their holdings 4.4. This component stores the user's portfolio and enables the risk metrics, performance analysis, stock news and diversification recommendations.

In this page, users can add, edit and delete stocks from their portfolio. When adding a stock, a pop-up to search stocks appears and once the user selects one, they must enter the shares and purchase price. Validation is performed on the stock data to ensure no null or invalid values were entered before it is sent to the backend for storage in the database.

The retrieval of the user's stock data is handled through API communication between the frontend and backend components.

Considering the user has already logged in, the process follows these steps, as shown in Fig 4.5:

1. User requests to view their stock holdings in the app.
2. Frontend sends a GET request to the backend API to fetch the user's holdings.
3. The backend validates the request format.

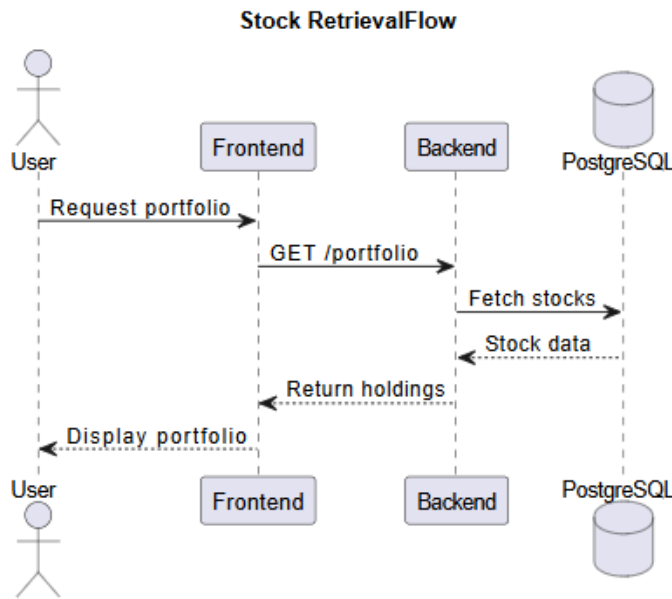


Figure 4.5: Diagram of the portfolio retrieval flow

4. Backend receives the request and queries the Database for all stocks associated with the user's ID.
5. Backend collects the stock holdings data and sends it as a Response back to the frontend.
6. Frontend displays the user's stock holdings to the User.

4.1.3 News Sentiment

The news sentiment page displays relevant financial news to the user with a sentiment label attached. Stock prices are influenced by many factors, and news is one of them, so it is crucial for the user to be aware of any happenings related to the company.

For this reason, a news page was implemented that fetches relevant financial articles from the Finnhub API. The LinearSVM model then applies sentiment classification to the articles and outputs a label of positive, negative or neutral for each.

News for each stock is displayed initially, but the user has the option to filter through news by stock.

The process follows these steps to generate sentiment labels for news, as shown in Fig 4.7:

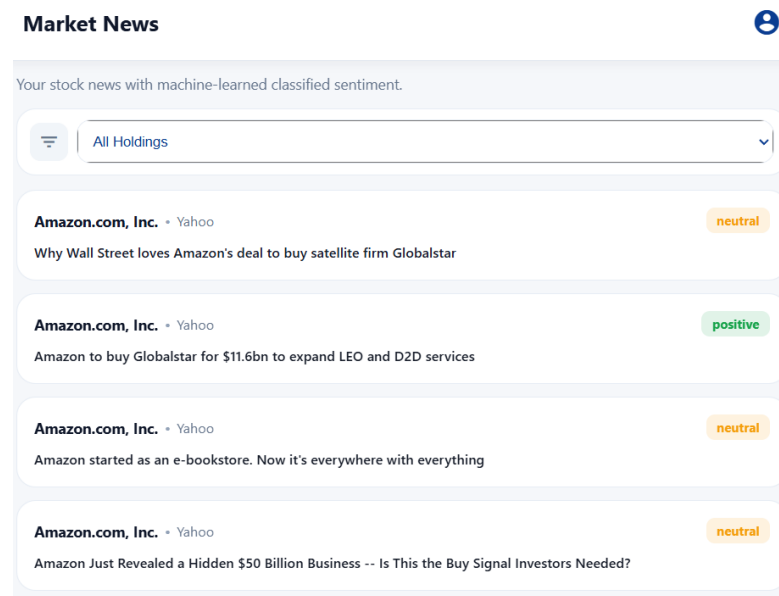


Figure 4.6: News page displaying financial articles with ML-classified sentiment label attached

1. User requests news and sentiment analysis in the app.
2. Frontend sends a request to the backend API for news and sentiment data.
3. Backend validates the request format.
4. Backend forwards the request to the FastAPI microservice.
5. FastAPI service calls the external news API (Finnhub) to fetch the latest news articles.
6. FastAPI processes the news articles and runs the model for sentiment analysis on the fetched news.
7. FastAPI returns the news articles along with sentiment scores to the Backend.
8. Backend sends the combined news and sentiment data as a Response to the Frontend.
9. Frontend displays the news and sentiment analysis to the User.

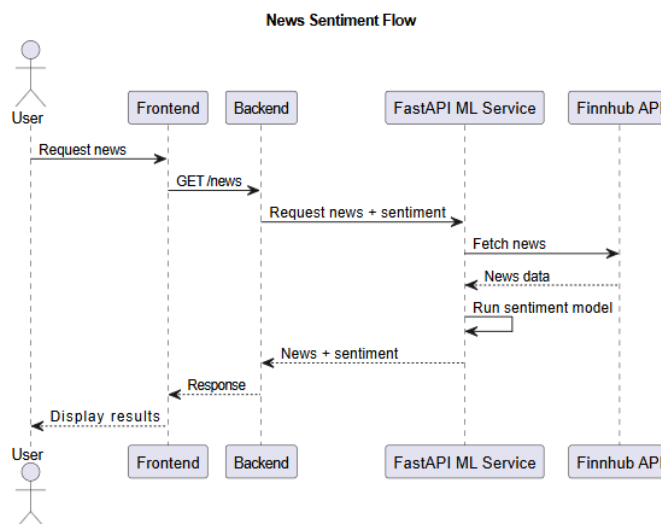


Figure 4.7: Diagram of news retrieval from Finnhub API and sentiment classification

News Filtering Pipeline

The pipeline operates directly on stock ticker symbols, which are used as the primary identifier when querying the Finnhub API. The following outlines the steps taken to retrieve and improve the quality of articles processed for sentiment analysis.

Company Name Split: The company name is shortened to its first word to only return headlines with its name in it

```

1  def split_company_name(company_name: str):
2  name = company_name.split('(')[0].strip()
3
4  name = name.split(',') [0].strip()
5  if '.' in name:
6      name = name.split('.')[0]
7  words = name.split()
8
9  return words[0] if words else company_name
  
```

Symbol News Retrieval: The stock symbol is used to query the Finnhub news endpoint for that particular stock's news+.

```

1  news_resp = requests.get(
2      "https://finnhub.io/api/v1/company-news",
  
```

```

3         params={
4             "symbol": symbol,
5             "from": from_date,
6             "to": to_date,
7             "token": api_key,
8         },
9         timeout=10,
10    )
11
12    articles = news_resp.json()

```

Date Filtering: Only the current day's news is used to retrieve the latest articles.

```

1    to_date = datetime.utcnow().date()
2    from_date = to_date

```

Headline Relevance Check: Only headlines with either company name or symbol are included for the most relevant articles.

```

1    def title_mentions_stock(headline, company_name, symbol):
2
3        headline_lower = headline.lower()
4        company_base = company_name.split('(')[0].strip().lower()
5
6        if symbol and symbol.lower() in headline_lower:
7            return True
8
9        if company_base and company_base in headline_lower:
10            return True
11
12    return False

```

Text Preparation: The headline and summary are combined into a single string for input to the sentiment model.

```

1    text = headline + " " + summary
2    article_texts.append(text)

```

Stock Sentiment

The stock sentiment page works in conjunction with the news sentiment service by computing the overall sentiment for each stock in the user's holdings. To do this, it calls the news retrieval function to get the stock articles, then accumulates all

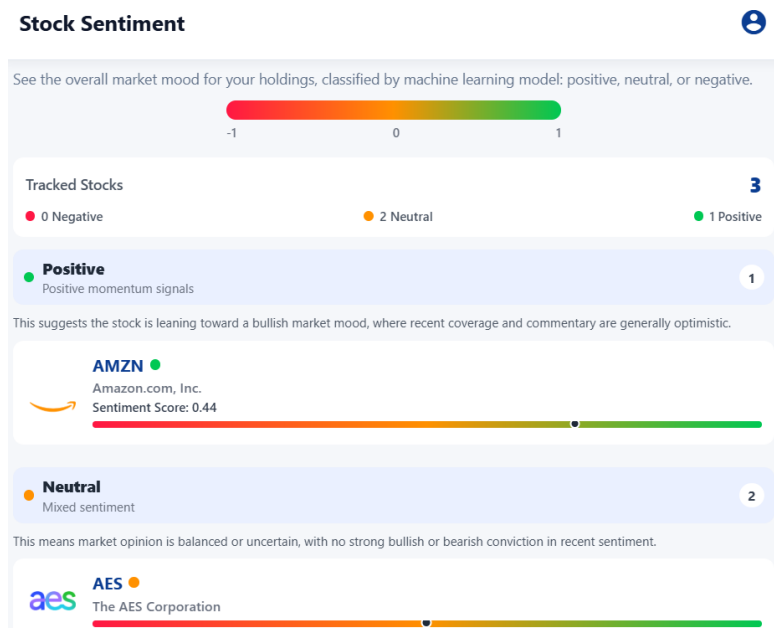


Figure 4.8: Stock Sentiment page showing the average sentiment score for each stock

the sentiment scores that were output by the LinearSVM model 2.8.1, to provide the average sentiment score for each stock. This page allows the user to quickly gauge the overall current sentiment for their holdings, as seen in Fig 4.8.

4.1.4 Analytics Script for Stock Features

This component is the base of the backend analytics layer. Unlike the sentiment analysis process, the portfolio analytics and diversification model do not execute on a per-request basis. Instead, there is a scheduled cron job on Render that runs a script daily to download stock data from Yahoo Finance. This script computes the key stock metrics (volatility, max drawdown, VaR, returns) and uses them as input for the stock risk classifications model. The results of this script are stored in a CSV file, which is then accessed by the FastAPI service when generating analytics for performance metrics, risk metrics, stock risk and stock recommendations. This design reduces the computational overhead. If it wasn't implemented, the application would need to download stock data and compute the diversification suggestions all in a single request. This design improves the application's response time.

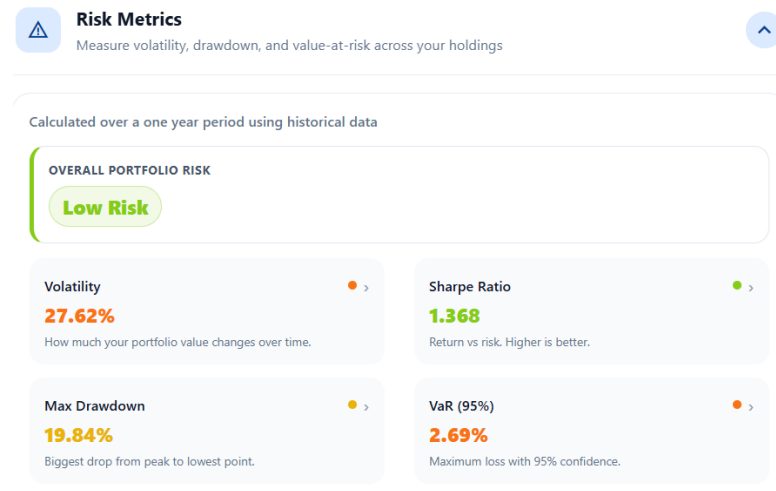


Figure 4.9: Portfolio level risk metrics showing Volatility, Max Draw Down, Sharpe Ratio, and VaR calculations

4.1.5 Risk Metrics

Using the precomputed features produced from the daily analytics script, discussed in ??, this service computes the portfolio and stock-level risk metrics of volatility, max drawdown and VaR using the calculations in 2.7. The output of this page is shown in Fig 4.9.

The risk metrics service also uses the analytics script output to return the user's stocks with its generated risk category attached. The model process defined in 2.8.2 clusters each stock into one of the following groups; very low risk, low risk, moderate risk, high risk, very high risk or extreme risk, as shown in 4.10.

This service communicates with the backend API to post the computed portfolio data back to the user in the frontend.

The risk metrics service follows this sequence of events, as shown in Fig 4.11 :

1. The user requests risk metrics in the frontend.
2. The frontend sends a POST request containing portfolio data to the Go backend API.
3. The backend validates the request and forwards the portfolio data to the FastAPI Risk Metrics Service.
4. The FastAPI service retrieves the precomputed stock features from the analytics dataset.

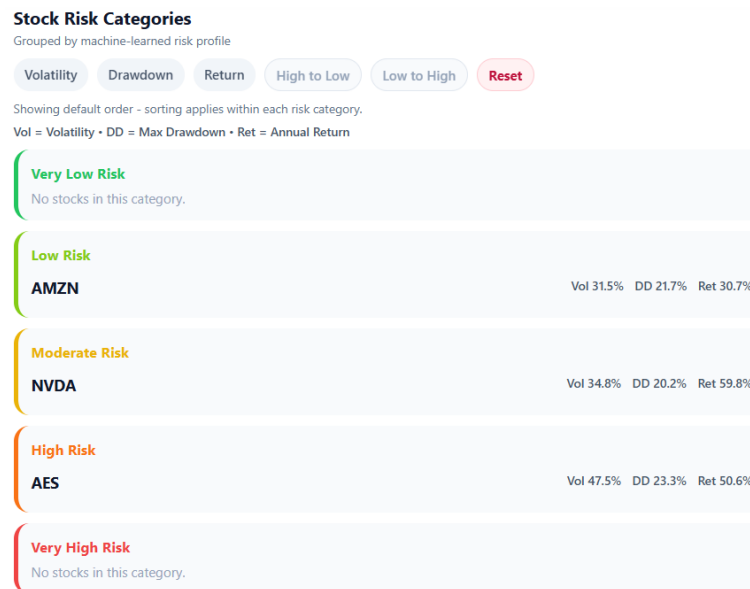


Figure 4.10: Stock risk clustering categories and stock level risk metrics

5. The FastAPI service computes portfolio-level metrics (volatility, maximum drawdown, Value at Risk, and Sharpe ratio).
6. The FastAPI service assigns risk categories to individual stocks using the GMM clustering results.
7. The FastAPI service determines the overall portfolio risk based on the distribution of stock risk categories.
8. The FastAPI service returns the response to the backend.
9. The backend sends the response to the frontend.
10. The frontend displays the results to the user.

4.1.6 Performance Metrics

The performance metrics service gives user's insights into the overall quality of their portfolio. Using the precomputed features produced from the daily analytics script ??, this service computes the user's total portfolio value, identifies the best and worst performing stock based on their returns, and computes the overall percentage returns for each stock to date, as shown in Fig ??.

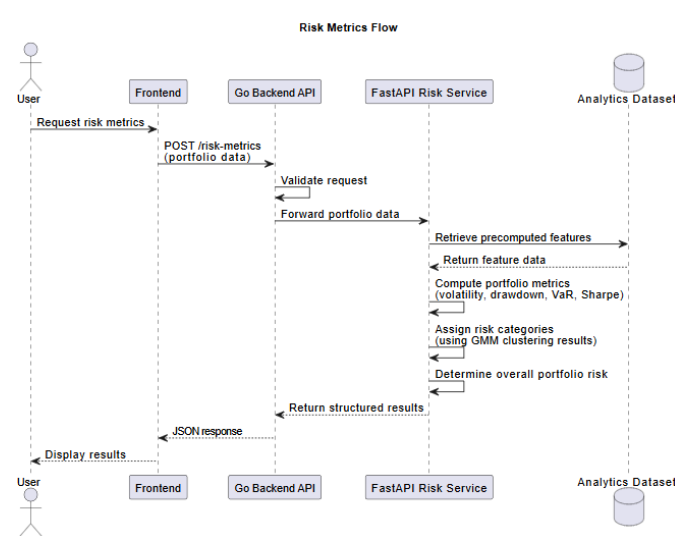


Figure 4.11: Risk metrics flow

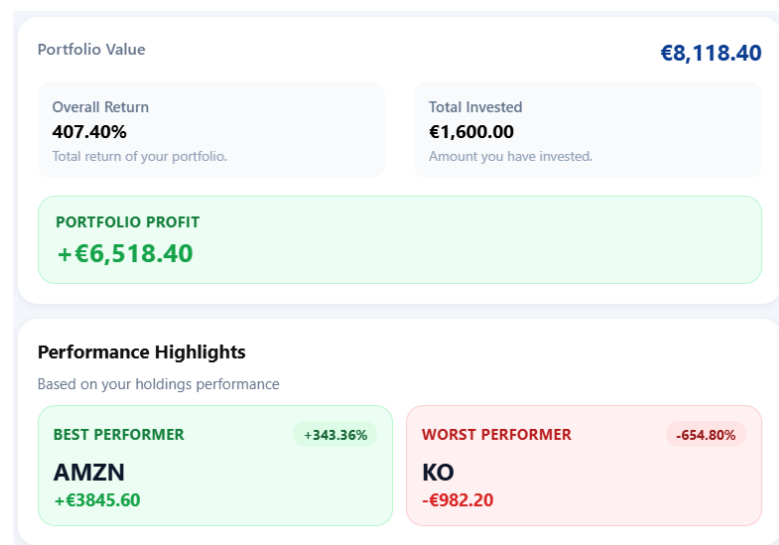


Figure 4.12: Portfolio Analysis page showing portfolio returns, best and worst performing stock, and holdings breakdown

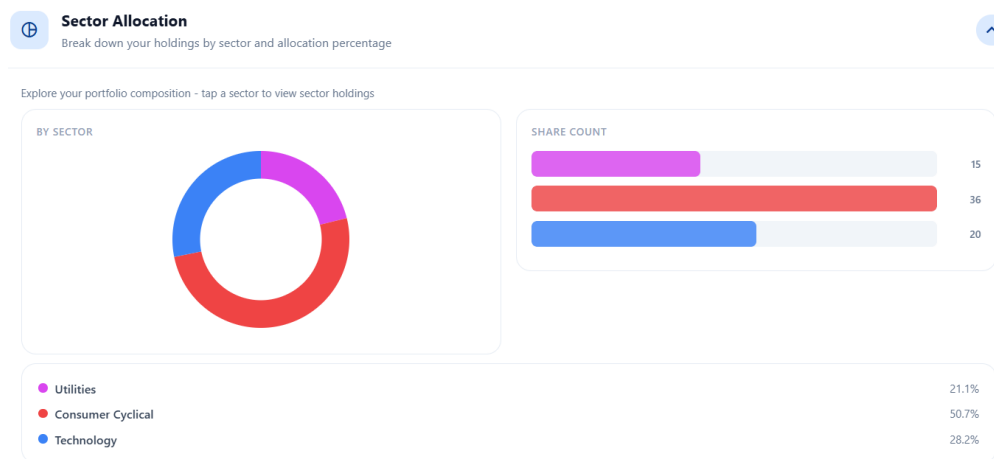


Figure 4.13: Sector allocation page displaying the interactive sector pie and doughnut chart

The process to fetch these features follows the same structure and design used for portfolio risk metrics, as described in 4.11.

4.1.7 Sector Allocation

The purpose of the stock allocation widget is to provide the user with a visual representation of all their stocks grouped by sectors. It visualises this through an interactive doughnut and bar chart with allocation percentages and share counts as shown in Fig 4.13.

This component requires communication with the backend API to retrieve the users' stocks to perform the computation to divide by sector.

4.1.8 Stock Recommendation

The purpose of the stock recommendation service is to provide the user with optimal stock suggestions that align with the risk tolerance they selected during account creation.

This feature works by analysing the user's current portfolio and identifying additional stocks that would optimise the portfolio's volatility in line with the user's risk preference.

The diversification process takes in the users' current stocks and their risk preference, to recommend stocks that best maximise the portfolio's volatility to suit their risk preference. This process does not just recommend individual stocks, but instead evaluates combinations of potential stocks to determine which set best

improves the overall portfolio.

The selected combination depends on the user's risk preference:

- Low risk: selects the combination with the lowest volatility
- High risk: selects the combination with the highest volatility
- Moderate risk: selects a combination closest to the median volatility

Fig 4.14 visualises the overall stock recommendation process.

1. User requests diversification suggestions in the app.
2. Frontend sends portfolio and risk preference to the Backend API.
3. Backend forwards the request to the Diversification FastAPI microservice.
4. The microservice processes the portfolio and applies the model.
5. Stocks are grouped into risk categories and filtered based on user preference, excluding already owned stocks.
6. The system tests combinations of stocks and estimates their impact on portfolio volatility.
7. The best combination is selected based on the user's risk level.
8. FastAPI returns recommendations to the Backend.
9. Backend sends results to the Frontend.
10. Frontend displays the suggestions to the User.

Stock Recommendation Pipeline

The following code snippets provide the main details of how this was implemented in the FastAPI stock recommendation service.

Combination Optimisation to Evaluate Portfolio Volatility

Listing 4.1: Combination search for candidate stocks

```
1 # Generate combinations of 1 to 3 candidate stocks
2 for r in range(1, 4):
3     combos.extend(combinations(candidate_symbols, r))
4
5 # Evaluate the volatility impact of each combination
```

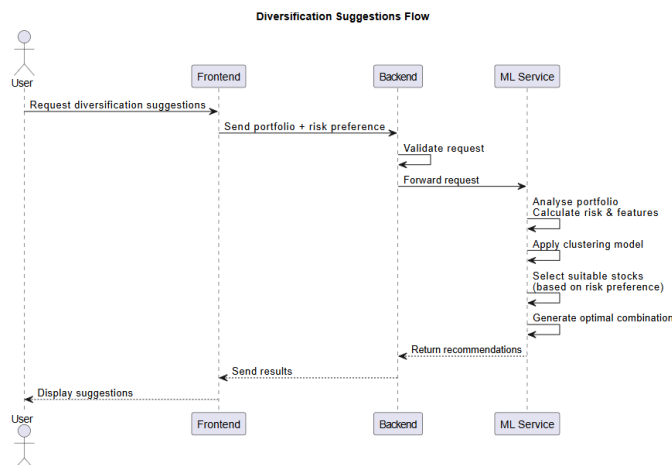


Figure 4.14: Diversification suggestions flow

```

6  for combo in combos:
7      projected = list(request.current_stocks)
8
9      for symbol in combo:
10         lookup_symbol = symbol.replace("-", ".")
11         company = STOCK_LOOKUP.get(lookup_symbol)
12
13         if not company:
14             continue
15
16         projected.append(
17             StockHolding(symbol=symbol, sector=company["
18                 sector"])
19         )
20
21     vol = calculate_portfolio_volatility(projected)
22
23     if vol is not None:
24         results.append({
25             "symbols": combo,
26             "volatility": vol
27         })

```

Selecting the Best Combination Based on User Risk Preference

Listing 4.2: Risk-based selection of recommended stocks

```

1  if request.user_risk_preference == "Low Risk":

```

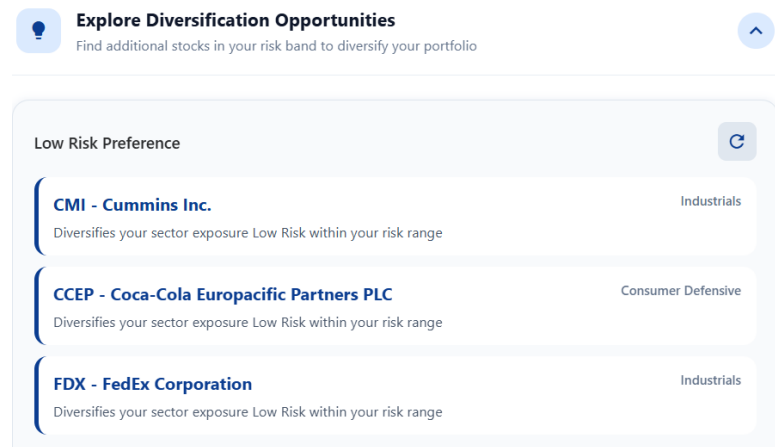


Figure 4.15: Example of the generated diversification suggestions in line with the user's selected risk preference

```

2     best = min(results, key=lambda x: x["volatility"])
3
4 elif request.user_risk_preference == "High Risk":
5     best = max(results, key=lambda x: x["volatility"])
6
7 else:
8     # Choose a volatility close to the median for moderate
      risk
9     vols = sorted(r["volatility"] for r in results)
10    median_vol = vols[len(vols) // 2]
11    best = min(results, key=lambda x: abs(x["volatility"] -
      median_vol))

```

4.1.9 Diversification Suggestions

The purpose of the diversification suggestions service is to provide users with additional stock recommendations by analysing their current portfolio to discover sectors they don't currently hold any stocks in. It sorts through the potential stocks to only suggest ones in line with the user's selected risk preference. This feature is evident in Fig 4.15. This service promotes portfolio diversification and reduced sector concentration.

This component involved interaction between the React frontend, Go backend API, database and the FastAPI service layer, as shown in Fig 4.16.

1. User requests stock suggestions from frontend.

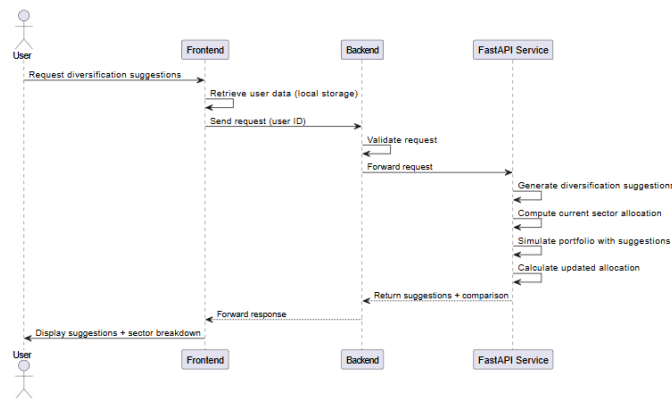


Figure 4.16: Diversification suggestions flow

2. Frontend retrieves user data from local storage.
3. Frontend sends request to backend API with user ID.
4. Backend validates request.
5. Backend forwards request to FastAPI recommendation service.
6. FastAPI generates random stock suggestions within the user's risk range.
7. FastAPI computes sector allocation for current portfolio.
8. FastAPI simulates portfolio with suggested stocks.
9. FastAPI calculates updated sector allocation.
10. FastAPI returns suggestions and portfolio comparison to backend.
11. Backend forwards response to frontend.
12. Frontend displays suggestions and sector breakdown to user.

4.1.10 Stock Risk

The stock risk checker service allows the user to check the risk level of any inputted stock that is available in the precomputed dataset from **??**. If the stock exists, it returns its risk category that was predicted by the GMM model and its individual stock risk metrics.

Once a user analyses the stock's risks, they can then simulate the effect adding a certain number of its shares will have on their overall portfolio. An example of

The screenshot displays the 'Stock Risk Checker' interface. At the top, a header bar contains a magnifying glass icon, the title 'Stock Risk Checker', and a subtitle 'Check a ticker's computed risk tier, volatility, and drawdown metrics'. Below this is a light blue box with the instruction 'Analyse the risk level of any stock ticker'. Inside this box, there is a text input field containing 'AAPL' and a dark blue 'Analyse' button. Below the input field, the results for 'AAPL - Apple Inc.' are shown, including a 'Very Low Risk' status and several metrics: Sector: Technology, Annualised Volatility: 23.79%, Max Drawdown: 13.8%, and Annual Returns: 30.94%. At the bottom of the results section, there is a 'Simulate Impact' section with the instruction 'See how adding this stock affects your portfolio before you purchase it with your desired shares'. This section includes a text input field labeled 'Number of shares' and a grey 'Simulate' button.

Stock Risk Checker
Check a ticker's computed risk tier, volatility, and drawdown metrics

Analyse the risk level of any stock ticker

AAPL

Analyse

AAPL - Apple Inc.
Very Low Risk
Sector: Technology
Annualised Volatility: 23.79%
Max Drawdown: 13.8%
Annual Returns: 30.94%

Simulate Impact
See how adding this stock affects your portfolio before you purchase it with your desired shares

Number of shares

Simulate

Figure 4.17: Enter Caption

this is visible in Fig ???. The portfolio simulation follows the same process as the risk metrics flow 4.11.

Both of these features follow the same process. The user send a request to the backend, the backend retrieves the used data, and the backend sends the data to the FastAPI service to compute the metrics.



Figure 4.18: Simulating the impact of 22 AAPL shares on this specific portfolio

Chapter 5

System Evaluation

In this chapter, I evaluate the final project implementation against the objectives that were defined in Chapter 1. The main goal of the application is to provide actionable insights to novice investors, to enable them to make more informed decisions in the stock market. StockWise does this by providing users with insights into the current state of their portfolio and giving feedback on how it can be improved to better align with their selected risk preference.

5.1 Evaluation Against Objectives

5.1.1 Objective 1: Secure User Account Creation

The first objective of the application was to provide secure account creation in order to enable personalised, user specific insights. This objective was met and is evident in 5.1. The system takes in the user's credentials, which includes a username, email and password and stores them in the database. In addition, the user's chosen risk preference is also stored during the account creation process, which enables the various insights and features.

To ensure security in account creation, the user's password is salted and hashed before it is stored in the database. Salting a password adds a unique string to the start of the user's password before it is hashed, while hashing converts the password into an encoded string that cannot be reversed. This approach strengthens the security of the application by preventing against common cyber attacks on passwords, such as dictionary and brute force attacks.

Acceptance testing was used to verify the implementation behaved as expected, as shown in 5.1.

A key strength of this implementation is that it provides a secure yet straightforward authentication process, suitable for the application. However, a limitation

Table 5.1: Secure Account Creation Acceptance Tests

Test case	Given	When	Then	Result
Valid user registration	A user provides valid registration details.	The form is submitted.	The user account is created successfully.	Pass
Duplicate email registration	An email already exists in the database.	The user submits the form.	The system rejects the request.	Pass
Invalid registration input	Required fields are missing or invalid.	The form is submitted.	Validation errors are shown.	Pass
Password hashing verification	A user registers successfully.	The password is stored.	The password is hashed, not stored in plain text.	Pass
Risk preference storage	A user selects a risk preference during registration.	The account is created.	The preference is stored correctly.	Pass

is that a multi-factor authentication approach was not implemented, which could further enhance the application's security and the user experience. Excluding this feature slightly reduces the quality of the process when compared against industry standards.

5.1.2 Objective 2: Portfolio Upload

The second objective stated was to provide a means for the user to upload their portfolio, which would enable the personalised features and insights based on their stock holdings. This was successfully implemented and can be viewed in the following figures 4.4. To upload their portfolio, the user must individually enter each stock holding they possess with its quantity and purchase price.

This implementation has validation against the user's inputs to ensure only valid digits are stored. The validation process ensures the user cannot enter null values or letters and only allows the user to add a stock once the fields are accurately filled in. This approach prevents unexpected data formats from being stored in the database.

Acceptance testing was conducted to verify the correctness of the portfolio upload behaviour to ensure it was as expected, as shown in table 5.2.

The main strength of this approach is the thorough error handling, which pre-

Table 5.2: Portfolio Upload Acceptance Tests

Test case	Given	When	Then	Result
Valid stock entry	A user enters a valid stock symbol, quantity, and purchase price.	The form is submitted.	The holding is saved successfully.	Pass
Missing input	One or more required fields are blank.	The form is submitted.	Validation errors are shown and nothing is saved.	Pass
Non-numeric quantity	The user enters letters in the quantity field.	The form is submitted.	The input is rejected.	Pass
Invalid price value	The user enters an invalid purchase price.	The form is submitted.	The input is rejected.	Pass
Duplicate holding	The same stock is already in the portfolio.	The user tries to add it again.	The system prevents duplication.	Pass

vents errors in the functionality of other features and ensures a smooth and seamless user experience. The downside of the implementation for portfolio uploading is that it may be time-consuming for users with large portfolios and can be seen as slightly impractical in that case. A more suitable approach for the application may be portfolio csv uploads instead of the manual additions, or to provide it as an option in conjunction with the current implementation.

5.1.3 Objective 3: ML Diversification Suggestions

The third objective outlined was to provide the user with ML driven diversification suggestions. This was implemented as the model generates risk-aware suggestions, and the outcome can be viewed in 4.15.

This process was achieved by using GMM clustering to group similar stocks based on their movement and performance in the stock market, taking both returns and volatility among other features into consideration. Through BIC and silhouette score, the discovered optimal number of clusters for the stock data was 6. This resulted in 6 categories a stock could be classified as; very low risk, low risk, moderate risk, high risk, very high risk, and extreme risk. The output of the stock clusters is visible in 5.1.

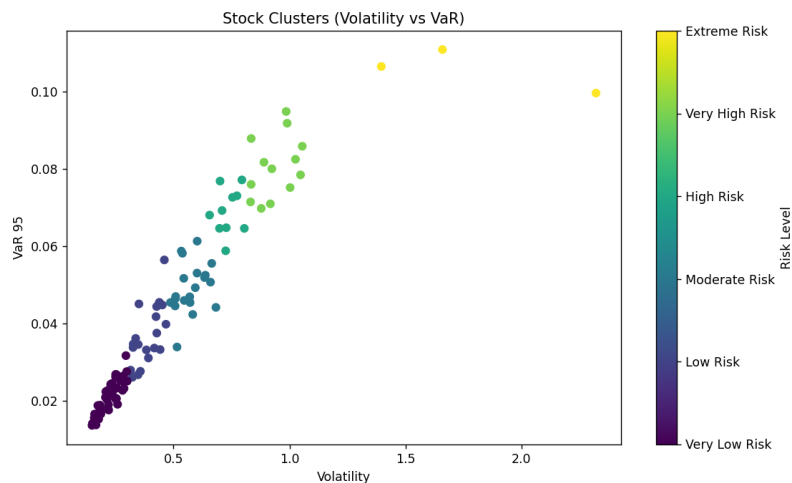


Figure 5.1: Stock risk clusters from the GMM model output

Model Performance

Bayesian Information Criterion (BIC) and silhouette score were used to evaluate the model across covariance types to decide the optimal clustering model. BIC is a statistical approach used to evaluate models based on how well it fits the training data with certain parameters, where lower values indicate a better balance. The silhouette score analyses how well separated the defined clusters are from each other, with values closer to 1 indicating good separation, values around 0 meaning there is some overlap and negative values indicating poor clustering.

Table 5.3 shows the best performing models for each covariance type based on BIC and silhouette. This output shows that the full covariance model with six clusters had the best BIC score of -507.05, meaning it has the best balance between parameters and fit. Even though the tied covariance model had a higher silhouette score, the model itself shows an overall poorer fit.

Table 5.3: Best Model per Covariance Type

Components	Covariance	BIC	Silhouette
6	full	-507.85575549354985	0.5004965967964337
4	tied	304.27248801707594	0.6121699175988665
6	diag	280.96937308786687	0.5105162108623956
5	spherical	309.2142825868415	0.5288603945821593

The stocks in each cluster were also assessed by their return and volatility to confirm the selected risk category matched the expected output based on their characteristics 5.4.

Table 5.4: Stock Risk Cluster Interpretation

Risk Level	Type	Description
Very Low	Defensive	Large, stable companies that don't change much in value.
Low	Stable Growth	Well-established companies with steady but not extreme growth.
Moderate	Mixed	Stocks that can go up or down more depending on the market.
High	Speculative	Riskier companies where prices can change a lot.
Very High	Thematic	Very volatile stocks, often driven by trends or hype.
Extreme	Outliers	Very unpredictable stocks with extreme price changes.

Using GMM clustering for risk-aware diversification suggestions worked well for the application due to its quick computation speeds, while still providing accurate results.

A shortcoming of this approach is its dependency on live stock data from Yahoo Finance. This is computationally expensive as it requires a daily update of stock data in order to provide accurate and up-to-date results.

5.1.4 Objective 4: Stock News Integration

The fourth objective of the application was to create a feature to display relevant financial news based on the user's stock holdings and to use ML to analyse each article to output a sentiment label. This was met, as articles relating to the user's stocks with the appropriate sentiment label attached, are evident on the news page. The implementation of this feature is demonstrated in 4.7.

The multi-step news filtering process outlined in 4.1.3 ensures that only articles relating to the user's holdings are processed by the model for classification. A sentiment label is produced when the combined article headline and summary are passed into the exported catboost ONNX model.

Model Performance

The sentiment classification model was evaluated using two different metrics. The first was the classification report, which consists of a precision, reclass, f1 and accuracy score. The precision score identifies the accuracy of correct predictions, while recall underscores the model's sensitivity. The F1-score is the harmonic

mean of the precision and recall scores, which provides a balanced metric. The accuracy, is its score against the predicted and actual values from the test set, which shows the number of correct predictions by the model.

The model's classification report, shown in Table 5.5 indicates the model has good performance with an overall accuracy score of 71%. The result highlights that the model performs best at predicting neutral labels with an f1-score of 78% and then negative labels with an f1-score of 73%. The most challenging class for the model to predict is positive with an f1-score of 64%. This is also evident in the confusion matrix 5.2, which shows that actual positives were often classified as negative and neutral.

Table 5.5: Classification Report for LinearSVC

Class	Precision	Recall	F1-score	Support
negative	0.73	0.71	0.72	832
neutral	0.76	0.80	0.78	833
positive	0.65	0.63	0.64	825
accuracy		0.71		2490
macro avg	0.71	0.71	0.71	2490
weighted avg	0.71	0.71	0.71	2490

A shortcoming of this feature is its dependency on the Finnhub API for financial news, as the quality and number of what is returned is limited by the sources the API has for a given stock on a given day. Another limitation of this feature is the model's challenge of predicting positive sentiment labels, evident in the low recall and precision scores. Using a sentiment classifier that can interpret text such as BERT may be a better fit for the implementation.

A key strength of this implementation is the ML inference speed as news is classified in a few seconds.

5.1.5 Objective 5: Risk Analytics and Feedback

The fifth objective in 1 was to provide users with risk analytics based on their stock holdings. This objective was achieved as the FastAPI risk service computes key risk metrics for the user's overall portfolio and individual stocks. The result of this implementation is displayed 5.4.

To calculate the overall risk profile of the user's holdings, the system uses weighted portfolio values. This means it takes into consideration the number of stocks the user holds for each share. So a stock with shared making up 80% of

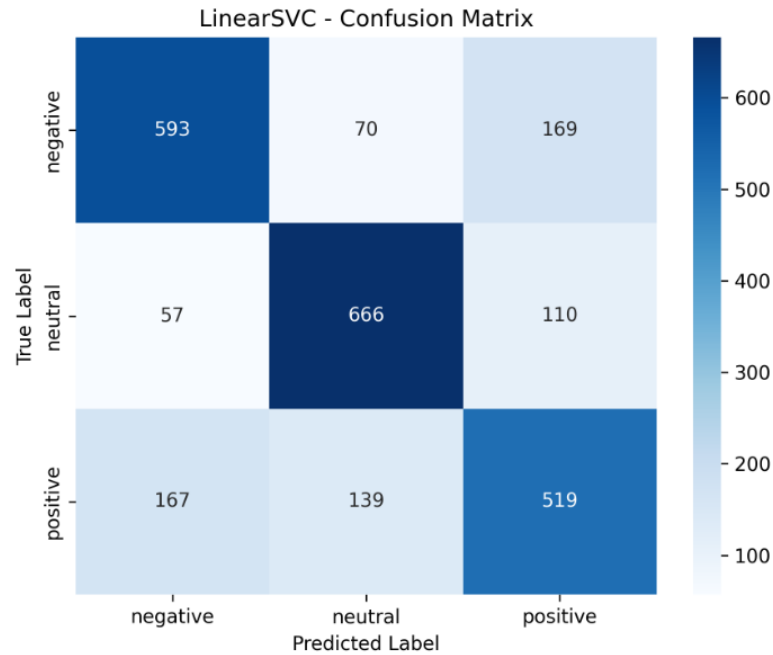


Figure 5.2: Confusion Matrix of sentiment classification model

the users portfolio will have more of an impact on the values, compared to a stock that only comprises 20%

To assess the correctness of the system, a series of manual testing and system testing was carried out to confirm that the expected outputs matched the actual ones.

Risk Metrics Testing

Table 5.6 is the test portfolio that was used to verify the correctness of the system, comprised of two stocks with different weights.

Table 5.6: Test portfolio composition

Ticker	Shares	Price	Value	Weight
AAPL	8	100	$8 \times 100 = 800$	$800/1000 = 0.80$
NVDA	2	100	$2 \times 100 = 200$	$200/1000 = 0.20$
Total			1000	1.00

Table 5.7 shows the series of manual testing and system outputs for comparison. The main strength of this implementation is that it provides realistic portfolio risk metrics by using the weighted values for each holding. Because of this, each

Table 5.7: Expected vs. actual portfolio values

Metric	Manual calculation (method)	System output	Match
Portfolio value	$800 + 200 = 1000$	1000.00	Yes
Weight AAPL	$\frac{800}{1000} = 0.80$	0.80	Yes
Weight NVDA	$\frac{200}{1000} = 0.20$	0.20	Yes
Annual return	Mean of $R_{p,t}$ over time $\times 252$	11.20%	Yes
Volatility	Std. dev of $R_{p,t} \times \sqrt{252}$	22.00%	Yes
VaR 95	5th percentile of $R_{p,t}$	-14.00%	Yes
Max drawdown	Minimum drawdown from returns	-23.60%	Yes

stock's contribution to the overall portfolio risk depends on its number of shares in the portfolio, rather than each stock being treated equally in the calculation. The main limitation of this feature is that the model only uses one year of historical data to evaluate the portfolio's risk, which may not accurately capture its overall trend in the market. To improve this, a longer historical time frame could be used.

5.1.6 Objective 6: Portfolio Performance Analysis

The final objective defined in 1 was to provide a performance analysis of the users' holdings, displaying critical information such as portfolio returns. This objective was met as the implementation computes the overall portfolio returns based on the user's average purchase price and the current price of each holding. The implementation also calculates the returns for each individual stock, further highlighting which ones drive profit or loss. This result can be viewed in ??.

To evaluate the correctness of the system, a series of manual testing and system testing was carried out to confirm that the expected and actual outputs matched.

Perfromance Analysis Testing

To calculate the return for each stock in the user's holdings and for the overall portfolio, the system uses the standard percentage change formula as shown in 2.7.

Table 5.8 shows the test data that was used as input to assess the correct value against the system implementation.

Table 5.9 shows the series of manual testing and system outputs for comparison. The main strength of the implementation is that it accurately computes the overall portfolio return of the user's holdings, as well as for each individual stock, so

Table 5.8: Test portfolio composition

Stock	Shares	P_0	P_t	Invested	Current
AAPL	10	180	202	$10 \times 180 = 1800$	$10 \times 202 = 2020$
NVDA	5	300	354	$5 \times 300 = 1500$	$5 \times 354 = 1770$
TSLA	2	350	332	$2 \times 350 = 700$	$2 \times 332 = 664$
Total				4000	4454

Table 5.9: Manual vs system portfolio performance results

Metric	Manual calculation	System output	Match
AAPL return	$\frac{2020-1800}{1800} = 0.12$	0.12	Yes
NVDA return	$\frac{1770-1500}{1500} = 0.18$	0.18	Yes
TSLA return	$\frac{664-700}{700} = -0.05$	-0.05	Yes
Total invested	$1800 + 1500 + 700 = 4000$	4000.00	Yes
Portfolio value	$2020 + 1770 + 664 = 4454$	4454.00	Yes
Overall portfolio return	$\frac{4454-4000}{4000} = 0.1135$	0.1135	Yes

it aligns with the objective of helping users understand their overall portfolio performance. A shortcoming of this feature is that benchmark returns are not provided.

5.2 Strengths and Limitations of the Overall System

This application is a useful decision support tool for novice investors, with its main strengths being its ease of use and robust error handling, security, as well as the combined financial analytics and ML outputs for portfolio insights. The validation steps taken when user input is required ensures data consistency between each layer of the application. Passwords being hashed and salted upon account creation and login provide an additional layer of security to prevent common password attacks.

The portfolio risk metrics are also realistic as they are calculated with weighted holdings, providing users with an accurate insight into their portfolio risk profile along with beginner-friendly explanations. The user's risk tolerance input allows the system to produce user-specific actionable insights in the features. The news and stock sentiment feature allows users to gauge market mood, providing context for the possible fluctuation of their holdings. The diversification model classifies stocks into the appropriate risk band, useful for novice investors to understand their current stock risk profile against their preferred risk tolerance.

The architectural design is also well suited to the application as different logic is contained to a specific layer. The frontend handles the user interface, while the backend API focuses on orchestration and routing and the FastAPI service provides the computations for the various insights. This separation of concerns makes the system easier to scale compared to monolithic architectures.

Despite the strengths, there are still limitations of the overall system. Portfolio entry must be entered manually, which could be considered inconvenient and time-consuming for investors with large portfolios. The risk metrics analytics only uses one year of historical data, which reduces computational overhead but may not fully capture a stock's risk patterns. The quality and number of articles returned from the FinnhubAPI depends on the sources it has for a given stock on a given day. And a ML model such as BERT may be more a more appropriate choice for sentiment classification.

Chapter 6

Conclusion

Retail investors often lack the tools necessary to analyse their stock portfolio, as opposed to institutional investors who have access to more resources. For beginner investors, this can be especially challenging as they may not have a strong financial background, leading them to make poor or miscalculated investment decisions based on gut feeling, social media or hype. Limited knowledge of portfolio risk management, market conditions, and the need for portfolio diversification exposes investors to unnecessary risk that can be easily mitigated.

The aim of StockWise is to provide a decision support tool for novice retail investors, to make smarter, more calculated investment decisions that are catered to their preferred risk preference. This was achieved by providing the user with actionable insights such as relevant stock news with sentiment, portfolio and stock level risk analytics, performance analysis, portfolio optimisation and diversification suggestions, as well as a portfolio simulator.

The System Evaluation chapter confirmed that each of the objectives outlined in the introduction were met.

- **Secure User Account Creation** Users can successfully create an account and their password is salted and hashed, adding a layer of security. Acceptance testing confirmed this process worked correctly.
- **Portfolio Upload:** Users can upload their portfolio by manually entering each stock, share and average purchase price, with validation used to guard against invalid entries. Acceptance testing confirmed this process worked correctly.
- **Machine Learning-Based Diversification Suggestions:** The Gaussian Mixture Model clusters stocks into six risk categories based on their volatility, VaR, and variance in the market, allowing risk-based diversification suggestions. BIC and silhouette score indicate the clusters have good separation.

- **Machine Learning News Sentiment Analysis:** The Logistic Regression model classifies news with a sentiment label of positive, negative or neutral. The classification report showed a model accuracy score of 73%.
- **Risk Analytics Dashboard:** Volatility, Var, and max drawdown are all computed accurately, as the manual calculations matched the ones generated by the system. The results are also displayed with beginner-friendly explanations to aid new investors.
- **Portfolio Performance Analysis:** Portfolio and stock returns are computed accurately as the manual calculations matched the ones generated by the system.

StockWise closes the gap for accessible investment tools for retail investors in the stock market through analytics and actionable insights, accompanied with beginner-friendly explanations to boost understanding.

Although each of the objectives was met, there are opportunities for further improvement in future work:

1. There is currently only one sign-up and login method: the addition of multiple authentication methods, such as signing in with Google or signing in with Apple or Face ID may be more convenient for the application especially for users on a mobile device.
2. The system also requires the addition of new stocks to optimise portfolio volatility for the user's risk preference: a meaningful improvement to the system would be the addition of another machine learning model that could understand the user's current holdings and risk preference, to generate updated portfolio weights and suggestions that align with their risk. For example, if a user's risk preference is low risk, the portfolio could generate suggestions such as reduce shares of X by 10% and increase shares of Y by 20% to optimise volatility. That way, investors can receive specific insights into how they can improve their current portfolio.
3. Data precomputed daily is used to generate the analytics for the features: further improvement could be made to the system if live data were used instead of the daily precomputed dataset for the metrics and insights. This limitation restricts investors to only uploading stocks that exist within the stock universe of the application when a broader spectrum would be more appropriate.

The implementation of StockWise achieved the goal of providing beginner investors with actionable insights to aid in their investment decision making process. This

was accomplished along with a sleek and modern user interface and an intuitive user experience in the application. StockWise is accomplished the goal of being accessible to all as it is available on all browsers regardless of device types.

Appendix A

Appendix

A.1 Source Code Repository

The source code for this project can be accessed on GitHub at: <https://github.com/JenniferOji/StockWise>

Bibliography

- [1] Chris Wheat and George Eckerd. A decade in the market: How retail investing behavior has shifted since 2015, August 2025. Research report. Available at: <https://www.jpmorganchase.com/institute/research/financial-health-wealth-creation/a-decade-in-the-market-how-retail-investing-behavior-has-shifted-since-2015>
- [2] Go comparative. Performance evaluation of programming languages as api services for cloud environments: A comparative study of php, python, node.js and go lang. In *2023 7th International Conference on Information Technology (InCIT)*, pages 293–297, 2023.
- [3] Athen Bozoglu. Understanding market dynamics: A strategic edge for institutional investors, 2025. Accessed: 28 December 2025. Available at: <https://www.nasdaq.com/articles/analytics/market-dynamics>.
- [4] Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952. Accessed: 17 November 2025. Available at: <http://www.jstor.org/stable/2975974>.
- [5] Ravi Jagannathan and Tongshu Ma. Risk Reduction in Large Portfolios: Why Imposing the Wrong Constraints Helps. *The Journal of Finance*, 58(4):1651–1683, August 2003. Accessed: 28 December 2025. Available at: <https://onlinelibrary.wiley.com/doi/10.1111/1540-6261.00580>.
- [6] Joy Dip Das, Sulalitha Bowala, Ruppa K. Thulasiram, and Aerambamoorthy Thavaneswaran. Portfolio Diversification with Clustering Techniques. In *2023 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 97–102, December 2023. ISSN: 2472-8322. Accessed: 30 November 2025. Available at: <https://ieeexplore.ieee.org/document/10371938>.
- [7] Stock holdings by u.s. households: Survey of consumer finances, 2022. Accessed: 28 November 2025. Available at: https://www.federalreserve.gov/econres/scf/dataviz/scf/chart/#series:Stock_Holdings;demographic:all;population:1;units:have.

- [8] Adam Hayes. Volatility: Meaning in finance and how it works with stocks, 2026. Accessed: 16 march 2026. Available at: <https://www.investopedia.com/terms/v/volatility.asp>.
- [9] David R. Harper. Understanding value at risk (var): Explanation and calculation methods, 2025. Accessed: 10 March 2026. Available at: <https://www.investopedia.com/articles/04/092904.asp>.
- [10] Jason Fernando. Sharpe ratio: Definition, formula, and examples, 2025. Accessed: 24 march 2026. Available at: <https://www.investopedia.com/terms/s/sharperatio.asp>.
- [11] Adam Hayes. Understanding maximum drawdown (mdd): Key insights and formula, 2025. Accessed: 18 march 2026. Available at: <https://www.investopedia.com/terms/m/maximum-drawdown-mdd.asp>.
- [12] Ankur Zing. Sentiment analysis for financial news, 2020. Accessed: 13 December 2025. Available at: <https://www.kaggle.com/datasets/ankurzing/sentiment-analysis-for-financial-news>.
- [13] Mac Lawson. Financial sentiment analysis notebook, 2023. Accessed: 27 December 2025. Available at: <https://www.kaggle.com/code/maclawson/financial-sentiment-analysis/input?select=data.csv>.
- [14] GitHub Datasets. S&p 500 companies dataset (constituents.csv), 2026. Accessed: 5 January 2026. Available at: <https://github.com/datasets/s-and-p-500-companies/blob/main/data/constituents.csv>.
- [15] Meta Platforms, Inc. React documentation, 2026. Accessed: 18 February 2026. Available at: <https://react.dev/>.
- [16] Mozilla Developer Network. Javascript, 2025. Accessed: 16 February 2026. Last modified: 2 October 2025. Available at: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.
- [17] Meta Platforms, Inc. Fetching data – react documentation, 2026. Accessed: 23 April 2026. Available at: <https://react.dev/learn/synchronizing-with-effects>.
- [18] Axios Contributors. Axios documentation, 2026. Accessed: 24 April 2026. Available at: <https://axios-http.com/docs/intro>.
- [19] The Go Authors. The go programming language documentation, 2024. Accessed: 9 February 2026. Available at: <https://go.dev/doc/>.

- [20] PostgreSQL Global Development Group. About postgresql, 2026. Accessed: 10 March 2026. Available at: <https://www.postgresql.org/about/>.
- [21] scikit-learn developers. sklearn.mixture.gaussianmixture, 2026. Accessed: 3 March 2026. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.mixture.GaussianMixture.html>.
- [22] Yue Li and Jianhua Wu. The estimation of var and es based on gaussian mixture model—take the shanghai composite index as an example. In *2022 22nd International Conference on Control, Automation and Systems (IC-CAS)*, pages 1497–1502, 2022.
- [23] scikit-learn developers. Support vector machines (svm), 2026. Accessed: 11 February 2026. Available at: <https://scikit-learn.org/stable/modules/svm.html>.
- [24] ONNX Project. Supported scikit-learn models (sklearn-onnx), 2026. Accessed: 2 March 2026. Available at: <https://onnx.ai/sklearn-onnx/supported.html>.
- [25] Fabian Pedregosa and Varoquaux. Scikit-learn: Machine learning in python. *J. Mach. Learn. Res.*, 12(null):2825–2830, November 2011. Available at: <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [26] scikit-learn developers. Getting started with scikit-learn, 2026. Accessed: 12 February 2026. Available at: https://scikit-learn.org/stable/getting_started.html.
- [27] scikit-learn developers. Gaussian mixture models — scikit-learn documentation, 2026. Accessed: 28 January 2026. Available at: <https://scikit-learn.org/stable/modules/mixture.html>.
- [28] Vercel Inc. Vercel documentation, 2026. Accessed: 17 February 2026. Available at: <https://vercel.com/docs>.
- [29] Render Inc. Render platform overview, 2026. Accessed: 11 February 2026. Available at: <https://render.com>.